LECTURE 26

MLPs for Signal Classification

From engineered DSP features to a trained neural classifier

75 minutes • architecture → cross-entropy → Adam → training loop

Senior + first-year graduate ECE

ROADMAP

What changes after Lecture 25?

0–20 min • Architecture

Assemble affine layers and ReLU into an MLP. Track tensor shapes from 8 DSP features to 3 classes.

20–45 min • Objective

Interpret logits, softmax and cross-entropy. Connect gradients to parameter updates and Adam.

45–75 min • Training loop

Walk through forward → loss → zero_grad → backward → step, then evaluate on held-out data.

Lecture 25 gave us tensors and ∇L. Lecture 26 answers: what computation do we differentiate, and how does it become a classifier?

DSP bridge: the inputs are still engineered features such as MFCCs, spectral flux, centroid, roll-off, ZCR and RMS.

DSP → ML

A signal becomes one feature vector per example

Example input vector

- spectral centroid
- zero-crossing rate
- spectral roll-off
- RMS energy
- spectral flux
- MFCC 1–3

One clip → 8 numbers → one class

input shape: (8,)

minibatch shape: (N, 8)

output logits: (N, 3)

Three signal classes

Cymbal • Kick • Snare

MLP ARCHITECTURE

The basic building block: an affine layer

$$\mathbf{y} = f(\mathbf{W}\mathbf{x} + \mathbf{b})$$

Weights

W mixes the input features. Every hidden unit receives a learned weighted combination of the previous layer.

Bias

b shifts the activation threshold. It lets the affine hyperplane move away from the origin.

Activation

f(·) supplies the nonlinearity that allows stacked layers to form complex decision regions.

If f is the identity everywhere, the network never escapes the family of affine maps.

SHAPE REASONING

Track the dimensions before touching the code

For one example

x: 8 features
W: 16 × 8
b: 16
z: 16 hidden pre-activations

For a minibatch of N examples

$$\mathbf{Z} = \mathbf{X}\mathbf{W}^T + \mathbf{1}\mathbf{b}^T$$

X: N × 8
Z: N × 16

$$N_{\text{params}} = D_{\text{out}}D_{\text{in}} + D_{\text{out}}$$

Linear(8,16) has 16·8 + 16 = 144 trainable parameters.

NONLINEARITY

ReLU keeps positive evidence and clips negative pre-activation

$$\mathrm{ReLU}(z) = \max(0, z)$$

Example

z = [−2, −0.5, 0, 1.2, 4]
ReLU(z) = [0, 0, 0, 1.2, 4]

Derivative

For $z < 0$ the local slope is 0; for $z > 0$ it is 1. The point $z = 0$ is handled by a chosen subgradient convention.

Why useful

Cheap, piecewise linear, and able to create different affine behavior in different regions of feature space.

Think “piecewise-linear decision surface,” not “mysterious biological neuron.”

CONCEPTUAL PROOF

Without a nonlinearity, depth collapses

$$\mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (\mathbf{W}_2\mathbf{W}_1)\mathbf{x} + (\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2)$$

Define $W_{eq} = W_2W_1$ and $b_{eq} = W_2b_1 + b_2$. Then the two-layer network is just one affine layer.

What depth alone cannot do

Two, ten, or one hundred linear layers still produce an affine decision boundary.

What ReLU changes

Each activation pattern selects a different local affine map, allowing a nonlinear overall boundary.

Good answer: “The composition of affine maps is affine; ReLU breaks that collapse.”

ARCHITECTURE

A deliberately small MLP for 8 DSP features

The final layer has 3 outputs because the task has 3 classes. These outputs are logits, not probabilities.

Total trainable parameters: (8·16+16) + (16·8+8) + (8·3+3) = 307.

Small enough to inspect • nonlinear enough to illustrate the full training machinery

PYTORCH

The forward model is ordinary tensor code

```
class SignalMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(8, 16),
            nn.ReLU(),
            nn.Linear(16, 8),
            nn.ReLU(),
            nn.Linear(8, 3),
        )

    def forward(self, x):
        return self.net(x)
```

Input

x has shape (N,8): one row per audio clip.

Output

logits has shape (N,3): one unconstrained score per class.

No softmax here

CrossEntropyLoss will handle the stable log-softmax internally.

CONCEPT CHECK

Pause & predict: shape reasoning

A minibatch contains 64 clips, each represented by 8 standardized DSP features. The first layer is Linear(8,16).

Question 1

What is the input tensor shape?

Question 2

What is the first weight-matrix shape?

Question 3

What is the hidden output shape?

Good answers: (64,8) • PyTorch stores Linear weight as (16,8) • hidden output is (64,16)

The network outputs logits: class scores before normalization

Example logits for one clip

$$z = [2.0, 1.0, 0.0]$$

A logit may be negative, positive, or large. It is not itself a probability.

Convert to probabilities only for interpretation

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

Softmax preserves ranking and produces positive values that sum to 1.

Key implementation rule

Feed raw logits to nn.CrossEntropyLoss(). Apply softmax afterward only when you want probabilities.

LOSS FUNCTIONS

Cross-entropy rewards high probability on the correct class

$$L = -\log p_y$$

$$L = -\frac{1}{N}\sum_{i=1}^{N} \log p_{i,y_i}$$

If p_y ≈ 1

Loss approaches 0. The model assigned high probability to the true class.

If p_y is small

The negative log becomes large. Confident wrong predictions are penalized heavily.

Why log?

Products of probabilities become sums of log-probabilities; gradients remain convenient and statistically meaningful.

Worked example: from logits to one loss value

$$\mathbf{z} = [2, 1, 0] \Rightarrow p_0 \approx 0.665 \Rightarrow L = -\ln(0.665) \approx 0.408$$

If class 0 is the correct label, the sample loss is about 0.408.

Interpretation

The network is leaning toward the correct class, but it is not yet extremely confident.

Good answer to “why not MSE?”

MSE can be used, but cross-entropy matches categorical likelihood and gives especially useful gradients for softmax classifiers.

PYTORCH DETAIL

CrossEntropyLoss wants class indices, not one-hot targets

For three classes

Cymbal → 0
Kick → 1
Snare → 2

Typical tensor shapes

logits: (N, 3)
labels: (N,)

```
criterion = nn.CrossEntropyLoss()
loss = criterion(logits, y_batch)  # y_batch dtype: torch.long
```

Common bug: applying softmax first, then passing probabilities to CrossEntropyLoss.

GRADIENT INTUITION

The loss gradient pushes the correct logit up and incorrect logits down

$$\frac{\partial L}{\partial z_k} = p_k - \mathbb{1}[k = y]$$

For the true class y, the term becomes p_y − 1. For every incorrect class, it is simply p_k.

True class

If p_y is too small, p_y − 1 is strongly negative. Gradient descent therefore tends to increase z_y.

Wrong classes

Positive p_k values cause gradient descent to reduce competing logits.

This is the local mechanism by which a categorical error becomes weight updates throughout the network.

OPTIMIZERS

Optimization turns gradients into parameter updates

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L$$

θ

All trainable weights and biases in the network.

η

Learning rate: how large a step we take.

∇θL

Direction of greatest local increase in loss; we move in the negative direction.

Too large η → unstable/overshooting. Too small η → painfully slow progress.

OPTIMIZERS

Adam adapts the update using gradient history

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t \qquad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \varepsilon}$$

First moment

m tracks an exponentially weighted average of recent gradients — momentum-like behavior.

Second moment

v tracks squared-gradient scale, allowing different parameters to get different effective step sizes.

Practical message

Adam is a strong default, not a guarantee. Learning rate and validation still matter.

PYTORCH

The optimizer owns the update rule; autograd owns the gradients

```
model = SignalMLP()
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(
    model.parameters(), lr=0.01
)
```

model.parameters()

Gives Adam the tensors it is allowed to update.

loss.backward()

Computes .grad values. It does not change parameters.

optimizer.step()

Reads those gradients and applies Adam’s update rule.

TRAINING LOOP

The five-step training idiom

1

Forward

logits = model(xb)

2

Loss

loss = criterion(logits, yb)

3

Clear

optimizer.zero_grad()

4

Backward

loss.backward()

5

Update

optimizer.step()

Conceptually: predict → measure error → forget stale sensitivity → compute current sensitivity → change parameters.

TRAINING LOOP

Step 1 — forward: compute class scores for the minibatch

```
logits = model(xb)     # shape: (B, 3)
```

At this point no parameter has changed. We have only evaluated the current model.

What happens?

- matrix multiplies + biases
- ReLU gates hidden activations
- final affine layer produces 3 logits
- autograd records the differentiable operations

Good debugging habit: assert logits.shape == (xb.shape[0], 3).

TRAINING LOOP

Step 2 — loss: turn many predictions into one scalar objective

```
loss = criterion(logits, yb)
```

$$L = -\frac{1}{N}\sum_{i=1}^{N} \log p_{i,y_i}$$

Why one scalar?

Reverse-mode autodiff is especially efficient when many parameters feed one scalar objective.

What the scalar summarizes

How much probability the model assigned to the correct class over the current minibatch.

Loss is a training objective; accuracy is an evaluation metric. They are related but not interchangeable.

TRAINING LOOP

Step 3 — clear old gradients before computing new ones

```
optimizer.zero_grad()
```

PyTorch accumulates gradients. If the previous minibatch left g_old in .grad, backward() will add g_new to it.

Intentional accumulation

Useful when you deliberately want several minibatches to simulate a larger effective batch.

Accidental accumulation

A common bug: updates become larger and depend on stale minibatches.

Good answer: zero_grad() resets the optimizer's parameter-gradient buffers.

TRAINING LOOP

Step 4 — backward: apply the chain rule through the entire network

```
loss.backward()
```

The graph is traversed from the scalar loss back through the output layer, ReLU gates, hidden layers, and eventually every trainable weight and bias.

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L$$

Important: backward() computes ∇θL and stores it in parameter .grad tensors. It does not perform the update.

TRAINING LOOP

Step 5 — step: let Adam change the weights

```
optimizer.step()
```

Adam reads the gradients now stored on the parameters, updates its moment estimates, and modifies the parameter values.

Before step()

parameters θ_t + gradients ∇θL

After step()

new parameters θ_{t+1}; gradients remain until cleared

A new forward pass is required to see predictions from the updated model.

IMPLEMENTATION

The complete minibatch training loop fits on a few lines

```
for epoch in range(num_epochs):
    model.train()

    for xb, yb in train_loader:
        logits = model(xb)
        loss = criterion(logits, yb)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

What repeats?

Every minibatch gets a fresh forward graph and a fresh gradient.

What persists?

The model parameters and Adam's optimizer state persist across updates.

What is an epoch?

One pass through the training examples, usually in shuffled minibatches.

VALIDATION / TEST

Evaluation is not training

```
model.eval()
with torch.no_grad():
    logits = model(X_test)
    pred = logits.argmax(dim=1)
    accuracy = (pred == y_test).float().mean()
```

model.eval()

Switches layers with train/eval behavior, such as dropout or batch normalization, into evaluation mode.

torch.no_grad()

Avoids recording gradient history when we only need predictions.

Held-out data

Do not tune repeatedly on the final test set. Use validation/CV for model choices.

DSP + ML HYGIENE

The network still depends on good experimental design

Standardize features

Centroid, ZCR, energy and MFCCs live on different numerical scales. Fit scaling on training data only.

Stratify the split

Preserve class proportions so train and held-out sets represent the same classification problem.

Beware leakage

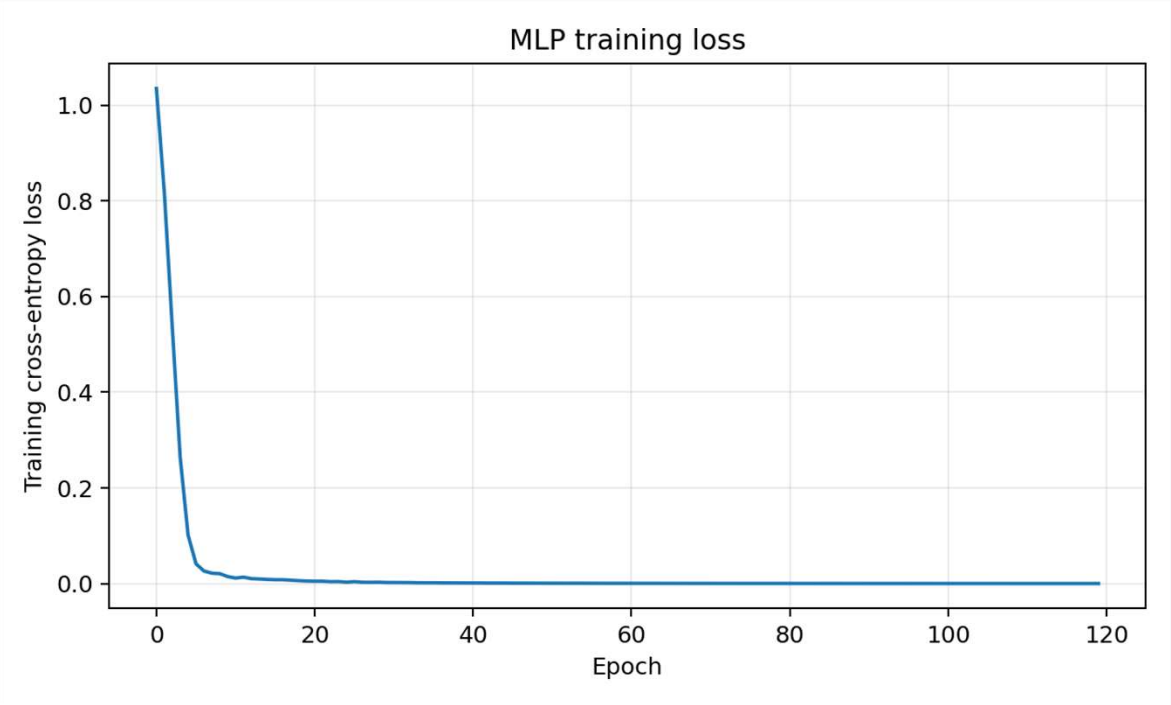
Frames from the same original recording should not be split across train and test if that lets recording-specific artifacts leak.

A neural network does not rescue a flawed data split. It may simply learn the leakage more effectively.

DSP question to keep asking: which variability belongs to the class, and which belongs to microphone, room, gain, performer, or session?

IN-CLASS RESULT

Demo: training loss falls as the MLP learns the synthetic classes



What to notice

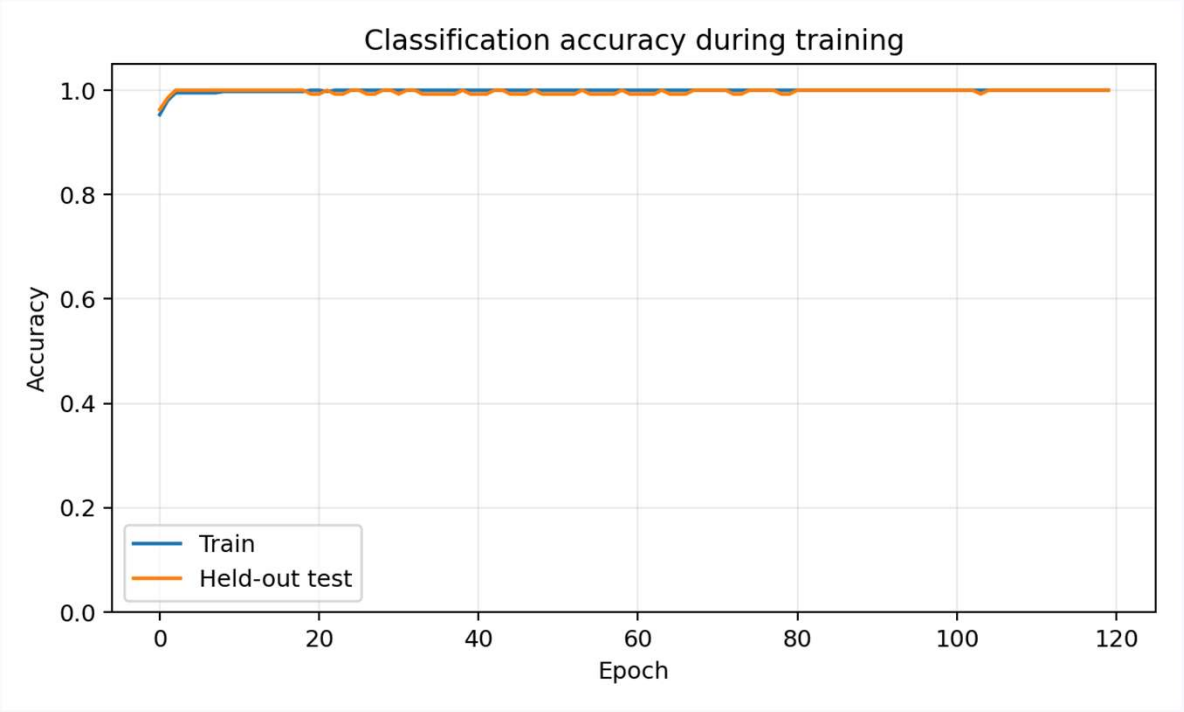
Large early improvement, then diminishing returns as the network fits the easy synthetic structure.

Do not overclaim

Low training loss alone does not prove generalization. We still need held-out evaluation.

IN-CLASS RESULT

Demo: compare training and held-out accuracy



Teaching dataset

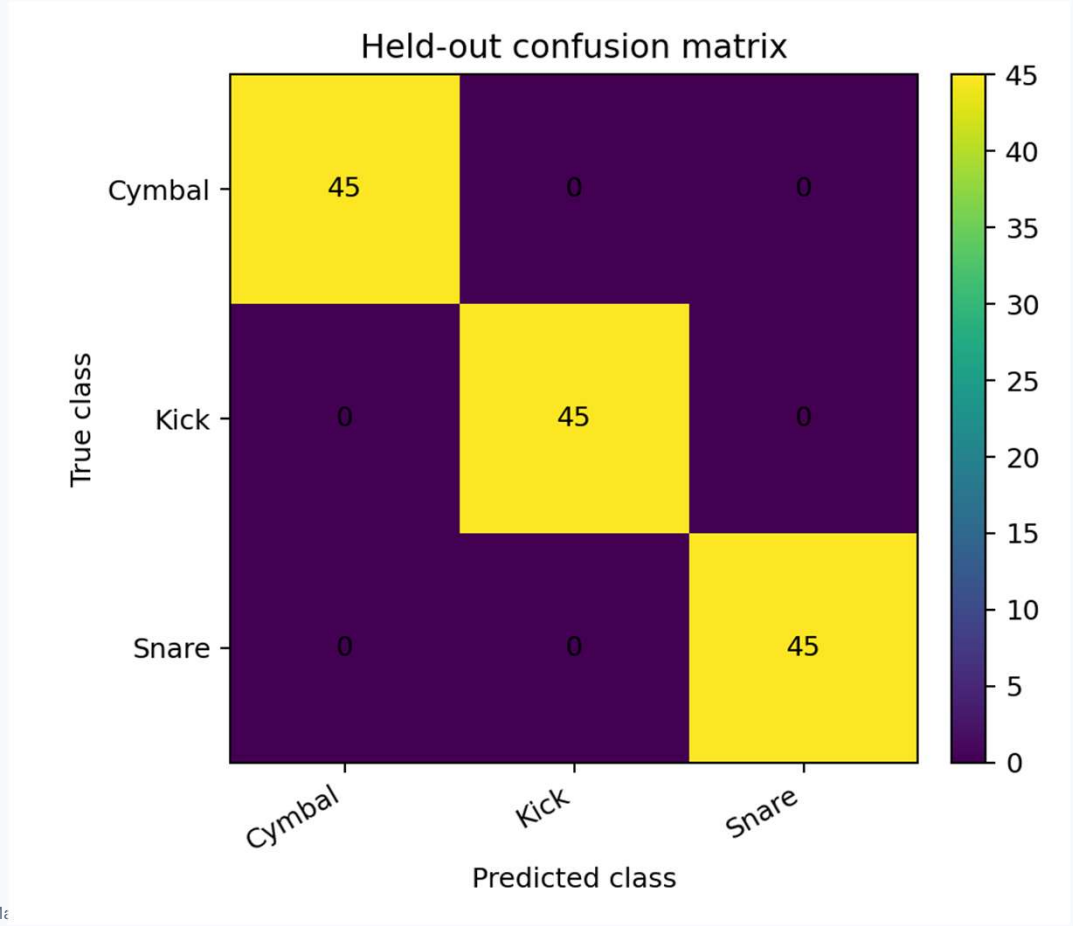
The synthetic Week 12 features are intentionally well separated, so accuracy can approach 100%.

Real audio

Expect overlap, domain shift and recording artifacts. Validation curves become more informative.

EVALUATION

Confusion matrix: which signal classes get confused?



Diagonal

Correct classifications. A strong diagonal is what we want.

Off-diagonal

Specific confusions tell us more than one accuracy number. Ask which physical features overlap.

In this synthetic example the classes are almost perfectly separable.

CONCEPT CHECK

Pause & diagnose the training loop

```
for xb, yb in loader:
    logits = model(xb)
    loss = criterion(logits, yb)
    loss.backward()
    optimizer.step()
```

What is missing?

optimizer.zero_grad()

What happens?

Gradients from old minibatches accumulate into the current parameter gradients.

Strong answer: “The code may still run, but the update no longer represents the gradient of just the current minibatch.”

WRAP-UP

Lecture 26 synthesis

Architecture

Affine layers + nonlinear activations turn DSP feature vectors into flexible decision functions.

Loss

Cross-entropy converts class predictions into a scalar learning objective.

Autograd

backward() computes gradients through the computation assembled in the forward pass.

Optimizer

Adam converts those gradients into parameter updates.

DSP features → standardize → minibatch → MLP → logits → cross-entropy → gradients → Adam → repeat

Minute paper: What is the conceptual difference between loss.backward() and optimizer.step()?

STUDENT VERSION

Homework — Problems 1–3

1 • Shapes

A minibatch has 64 examples with 8 DSP features. First hidden layer has 16 neurons. Give input shape, first weight/bias shapes, and hidden output shape.

2 • Why ReLU?

Show algebraically that two affine layers with no nonlinear activation are equivalent to one affine layer. Explain the consequence for decision boundaries.

3 • ReLU

For $z = [-2, -0.5, 0, 1.2, 4]^T$, compute ReLU(z) and state its derivative for $z<0$ and $z>0$.

STUDENT VERSION

Homework — Problems 4–7

4 • Cross-entropy

Logits $z=[2,1,0]$, true class 0. Compute the softmax probability of class 0 and the sample cross-entropy.

5 • Update

Given $\theta=1.5$, $\eta=0.02$ and $\partial L/\partial\theta=4$, compute one gradient-descent update.

6 • Training loop

Explain the purpose and order of the five PyTorch steps. What fails if zero_grad() is omitted?

7 • DSP scaling

Explain why standardization helps an MLP using centroid, ZCR, energy, flux, roll-off and MFCCs.

10 MINUTES

Weekly quiz — Questions 1–3

Q1 • Shape

A batch contains 32 examples, each with 8 DSP features. What is the input shape to Linear(8,16)?

Q2 • Nonlinearity

Why are two consecutive affine layers with no nonlinear activation equivalent to one affine layer?

Q3 • CrossEntropyLoss

Should the final network layer apply softmax before nn.CrossEntropyLoss()? Briefly justify.

10 MINUTES

Weekly quiz — Questions 4–6

Q4 • Loss

The model assigns probability p_y=0.8 to the correct class. Compute the sample cross-entropy.

Q5 • backward()

What does loss.backward() compute?

Q6 • step()

What is the conceptual difference between loss.backward() and optimizer.step()?

INSTRUCTOR SOLUTION APPENDIX

Homework + Quiz Solutions

Keep after the student-facing material or move into a separate instructor deck.

INSTRUCTOR APPENDIX

Quiz solutions — Questions 1–3

A1

(32,8). Thirty-two examples; eight features per example.

A2

The composition of affine maps is affine: $W_2(W_1x+b_1)+b_2 = (W_2W_1)x + (W_2b_1+b_2)$. Without a nonlinearity, depth does not create nonlinear boundaries.

A3

No. Return raw logits. CrossEntropyLoss internally applies a numerically stable log-softmax plus negative log-likelihood.

INSTRUCTOR APPENDIX

Quiz solutions — Questions 4–6

A4

$L = -\ln(0.8) \approx 0.223$.

A5

backward() computes gradients of the scalar loss with respect to upstream learnable parameters and stores them in .grad.

A6

backward() computes gradients. step() applies Adam's update rule to change parameter values using those gradients.

INSTRUCTOR APPENDIX

Homework solutions — Problems 1–3

1 • Shapes

Input: (64,8). Linear(8,16) weight: (16,8), bias: (16,). Hidden output: (64,16).

2 • Linear collapse

Substitution gives $y = (W_2W_1)x + (W_2b_1+b_2)$, which is one affine map. ReLU is needed to create nonlinear piecewise-affine behavior.

3 • ReLU

ReLU(z) = [0,0,0,1.2,4]^T. Derivative is 0 for $z<0$ and 1 for $z>0$; behavior exactly at zero is handled by a subgradient convention.

INSTRUCTOR APPENDIX

Homework solutions — Problems 4–7

4 • Cross-entropy

$p_0 = e^2/(e^2+e^1+e^0) \approx 0.6652$. Therefore $L = -\ln(p_0) \approx 0.4076$.

5 • Update

$\theta_{next} = 1.5 - 0.02(4.0) = 1.42$.

6 • Loop

Forward → loss → clear → backward → step. Omitting zero_grad() causes unintended accumulation across minibatches.

7 • Scaling

Comparable numerical scales improve optimization conditioning. Fit mean/std on training data only to avoid leakage.